



17.1. The Transformer Models

Understanding language by learning what every word should pay attention to.

Previous Section:

 [17. Neural Networks for Texts](#)

Contents:

[1. Introduction to Transformer](#)

[2. The Transformer Architecture](#)

[Step 1. Input Processing](#)

[Step 2. Embedding and Transformer Layers](#)

[Step 3. Output Layer](#)

[The Heart of the Transformer: Self-Attention](#)

[Computing Attention](#)

[Multi-Head Attention](#)

[3. Transformer Encoder Block](#)

[3.1. Residual Connections](#)

[3.2. Layer Normalization](#)

[3.3. Why Do We Need Positional Embeddings?](#)

[3.4. Implementing Scaled Dot-Product Attention](#)

[3.5. Building a Simple Transformer Encoder Block](#)

[Summary](#)

[References](#)

In the previous section, we studied how Neural Networks can process text using such a architecture like ***Bag-of-Words Sentiment Classifier***. While these

models were revolutionary for their time, they also introduced several limitations, especially when working with long sequences of text.

In this section, we explore one of the most influential breakthroughs in modern Artificial Intelligence: the **Transformer**. Introduced in 2017, the Transformer fundamentally changed how we process language by replacing recurrence with a mechanism known as **self-attention**. Today, nearly every state-of-the-art language model—including ChatGPT, Gemini, Claude, and many others—is built upon the Transformer architecture.

We will first understand how Transformers work, before implementing a **Transformer Encoder** for sentiment classification on the IMDB dataset and a **Transformer Decoder** for machine translation.

1. Introduction to Transformer

For many years, Recurrent Neural Networks (RNNs) and their variants such as LSTMs were the dominant models for Natural Language Processing (NLP). Since words naturally appear in sequence, it seemed reasonable to process one word after another.

However, in 2017, a research paper titled ***“Attention Is All You Need”*** introduced a completely different architecture known as the **Transformer**. Rather than processing words sequentially, Transformers rely almost entirely on an attention mechanism to model relationships within a sequence.

This seemingly simple idea turned out to be revolutionary.

Instead of reading one token after another, Transformers can process every token in a sentence simultaneously. This makes training significantly faster while also allowing the model to capture long-range relationships much more effectively than traditional RNNs.

As a result, Transformers rapidly replaced recurrent architectures in most NLP tasks, including: Sentiment analysis; Machine translation; Text summarization; Question answering; Large Language Models (LLMs)

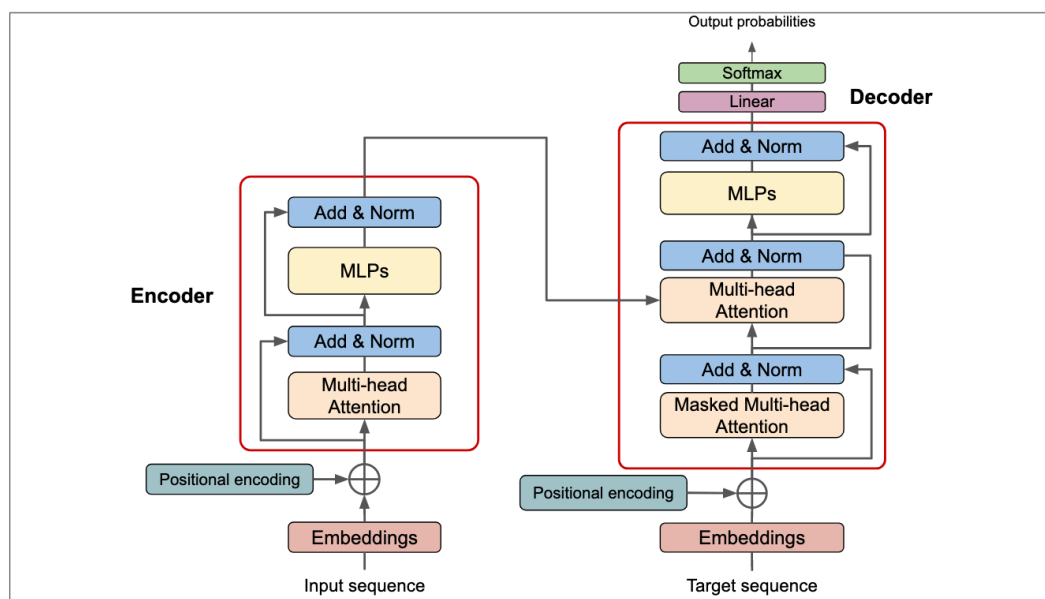
Throughout this section, we will implement two common applications of this architecture:

- A **Transformer Encoder** for IMDB movie review classification.
- A **Transformer Decoder** for machine translation.

Before writing any code, however, we first need to understand how the Transformer architecture is organized.

2. The Transformer Architecture

Although Transformer models may appear complicated at first glance, their overall architecture can be understood as three major components working together.



Step 1. Input Processing

Everything begins with raw text. Since computers cannot directly understand words, we first tokenize the sentence into smaller units called **tokens**. These tokens are then converted into numerical IDs that can be represented as tensors.

For example: "The movie was amazing." → ["The", "movie", "was", "amazing", "."] → [101, 2023, 2001, 6429, 102]

These numerical representations become the input to the Transformer.

Step 2. Embedding and Transformer Layers

Next, the token IDs pass through an **Embedding Layer**, which converts each token into a dense vector representation.

These embeddings are then processed by multiple **Transformer Layers**, the true computational core of the model.

Each Transformer layer mainly consists of two building blocks:

- A **Self-Attention** mechanism
- A **Feed-Forward Neural Network (FFN)**

Depending on the task, these layers are organized differently.

- **Transformer Encoders** are designed to understand the meaning of an input sequence.
- **Transformer Decoders** generate new tokens one at a time based on previously generated tokens.

The quality of a Transformer model is largely determined by how effectively these layers learn relationships between different parts of the input.

Step 3. Output Layer

Finally, the transformed representations are passed to an output layer that depends on the task. For example,

- A classifier predicts whether an IMDB review is positive or negative.
- A translation model predicts the next word in another language.
- A language model predicts the next token in a sentence.

Although these tasks differ, they all rely on the same Transformer backbone.

The Heart of the Transformer: Self-Attention

Among all components of the Transformer, **self-attention** is by far the most important.

It is the mechanism that allows every token in a sentence to examine every other token and determine which ones are most relevant when building its own representation.

Instead of assigning every surrounding word equal importance, self-attention learns to give higher importance to informative words while reducing the influence of less relevant ones.

In a way, this resembles ideas we have already encountered.

- **TF-IDF** increases the importance of informative words while reducing the weight of common words.
- **Max Pooling** keeps only the strongest activations.

These methods can be viewed as simple forms of selecting important information.

Modern attention mechanisms extend this idea much further by producing **context-dependent representations**, meaning that the representation of a word changes depending on the words surrounding it.

For example, consider the sentence: **"The train left the station on time."**

After tokenization, each word initially receives its own embedding vector.



Without context, the word **"station"** has only one fixed representation.

However, the meaning of **"station"** in **"train station"** is clearly different from its meaning in **"radio station"**.

Self-attention solves this problem by allowing the token **station** to examine nearby words such as **train**, giving them higher importance when constructing its new representation. As a result, every word becomes **context-aware**, rather than keeping a single fixed meaning.

Computing Attention

Conceptually, attention measures how strongly one token should focus on another.

- A simple similarity score between two tokens can be written as $s_{ij} = x_i^T x_j$, where x_i and x_j are the vector representations of two tokens.
- These scores are normalized using the **Softmax** function, producing weights whose sum equals one.
- Finally, a weighted combination of the token vectors is computed, allowing more important words to contribute more strongly to the final representation.

Modern Transformers extend this idea using three learned matrices: **Query (Q)**; **Key (K)**; **Value (V)**. For each token: the **Query** asks what information is needed, the **Keys** describe what information every token contains, and the **Values** store the actual information that will be combined.

Each Query is compared against every Key to produce attention scores, which determine how much of each Value contributes to the output.

The standard attention equation is

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

where d_k is the dimension of the key vectors. This equation is the mathematical foundation of nearly every modern Transformer model.

Multi-Head Attention

One attention mechanism is powerful, but it may only capture one type of relationship between words. To overcome this limitation, Transformers use **Multi-Head Attention**.

Instead of computing a single attention map, the model creates several independent attention heads.

- Each head learns its own projections of the Query, Key, and Value matrices, allowing it to focus on different aspects of the sentence.

- For example, one head may learn grammatical relationships, another may capture semantic similarity, while another focuses on long-range dependencies.
- The outputs of all heads are then concatenated and projected once more to produce the final representation.

By observing multiple perspectives simultaneously, Multi-Head Attention enables Transformers to capture much richer relationships than a single attention mechanism alone.

3. Transformer Encoder Block

Now that we understand how self-attention works, we can finally assemble the **Transformer Encoder Block**, which is the fundamental building block of encoder-based models such as **BERT** and many text classification networks.

A Transformer encoder repeatedly applies the same sequence of operations to gradually transform simple word embeddings into richer contextual representations.

A standard encoder block consists of four major components:

- Multi-Head Self-Attention
- Feed-Forward Neural Network (Dense Projection)
- Residual Connections
- Layer Normalization

Rather than replacing the original information after each operation, the encoder continuously refines it while preserving useful features from previous layers.

3.1. Residual Connections

One problem with very deep neural networks is that information can gradually disappear as it passes through many layers. Likewise, gradients become weaker during backpropagation, making the network difficult to train.

To alleviate this problem, Transformers use **Residual Connections**. Instead of only passing the output of a layer forward, we also add the original input back to the output.

Mathematically, $y = F(x) + x$. Where: x is the original input; $F(x)$ is the transformation performed by the layer.

This simple addition allows important information to flow directly through the network while also making gradient propagation much easier. As a result, modern Transformers containing dozens—or even hundreds—of layers remain trainable.

3.2. Layer Normalization

After each residual connection, Transformers apply **Layer Normalization**. Earlier neural networks often used **Batch Normalization**, which computes statistics across an entire mini-batch.

This works extremely well for many computer vision tasks, where images within the same batch generally have similar statistical properties.

Sequence data, however, behaves differently. Sentences have different lengths, different meanings, and different distributions. Instead of normalizing across the batch, Layer Normalization normalizes each sequence independently over its feature dimension.

Conceptually, $\hat{x} = \frac{x - \mu}{\sigma}$, where the mean and standard deviation are computed over the feature axis of a single sample.

Because every sentence is normalized independently, Layer Normalization is considerably more stable than Batch Normalization for sequence modeling, making it the standard normalization technique in modern Transformers.

3.3. Why Do We Need Positional Embeddings?

There is one subtle problem with self-attention. Self-attention treats its input as a **set** of vectors. By itself, it has no notion of order.

For example, “The dog chased the cat.” and “The cat chased the dog.” contain exactly the same words, yet their meanings are completely different.

Without positional information, self-attention cannot distinguish between these two sentences.

To solve this problem, Transformers add a **Positional Embedding** to every word embedding before entering the encoder.

Instead of representing a token only by its word embedding w_i , we combine it with a learned positional embedding p_i : $z_i = w_i + p_i$. The resulting vector now contains both the meaning of the word, and its position within the sentence.

In the earlier section, we already implemented a **PositionalEmbedding** layer. Since it performs exactly the operation required by Transformers, we can simply reuse it here.

If you have not implemented it previously, the implementation is shown below.

```
class PositionalEmbedding(layers.Layer):
    def __init__(self, sequence_length, input_dim,
                  output_dim, **kwargs):

        super().__init__(**kwargs)

        self.token_embeddings = layers.Embedding(
            input_dim=input_dim,
            output_dim=output_dim,
            mask_zero=True,
        )

        self.position_embeddings = layers.Embedding(
            input_dim=sequence_length,
            output_dim=output_dim,
        )

    def call(self, inputs):
        length = tf.shape(inputs)[1]
        positions = tf.range(start=0, limit=length, delta=1)

        token_embeddings = self.token_embeddings(inputs)
        position_embeddings = self.position_embeddings(positions)

        return token_embeddings + position_embeddings
```

```
def compute_mask(self, inputs, mask=None):
    return self.token_embeddings.compute_mask(inputs)
```

3.4. Implementing Scaled Dot-Product Attention

Before building an encoder block, let us first implement the core attention mechanism ourselves using NumPy.

This implementation follows exactly the mathematical equation introduced earlier.

The algorithm consists of four steps:

1. Compute the similarity between every Query and Key.
2. Scale the scores by $\sqrt{d_k}$.
3. Apply the Softmax function to obtain attention weights.
4. Multiply the attention weights with the Value matrix to obtain the final context vectors.

```
def scaled_dot_product_attention(query, key, value):
    # Calculate the attention weights based on scaled dot product attention.
    # Compute QKT
    matmul_qk = np.matmul(query, np.transpose(key, (0, 2, 1)))

    # Scale
    dk = key.shape[-1]
    scaled_attention_logits = matmul_qk / np.sqrt(dk)

    # Softmax
    attention_weights = tf.nn.softmax(scaled_attention_logits,
                                      axis=-1).numpy()

    # Weighted sum
```

```
output = np.matmul(attention_weights, value)

return output, attention_weights
```

To see how attention behaves, we create a simple sentence and generate random Query, Key, and Value matrices.

```
sentence = ["the", "train", "left", "the", "station", "on",
            "time"]

seq_len = len(sentence)
d_model = 4

np.random.seed(42)

query = np.random.rand(1, seq_len, d_model)
key   = np.random.rand(1, seq_len, d_model)
value = np.random.rand(1, seq_len, d_model)

attention_output, attention_weights = scaled_dot_product_at
tention(query, key, value)

print("Attention Output shape:", attention_output.shape)
print("Attention Weights shape:", attention_weights.shape)

print("\nAttention Output:\n", attention_output[0])
print("\nAttention Weights:\n", attention_weights[0])
```

```
Attention Output shape: (1, 7, 4)
Attention Weights shape: (1, 7, 7)
```

Although the vectors are randomly generated, the example illustrates how every token produces an attention distribution over all other tokens in the sentence.

In a real Transformer, these Query, Key, and Value matrices are learned automatically during training.

3.5. Building a Simple Transformer Encoder Block

Now we can combine everything into a simplified Transformer encoder block. To keep the implementation easy to understand, this example includes: Self-Attention; Feed-Forward Network; Residual Connections

For simplicity, the Layer Normalization step is omitted, since implementing it manually with NumPy requires considerably more code. In practice, modern Transformer libraries such as TensorFlow and PyTorch always include Layer Normalization inside every encoder block.

```
class SimpleTransformerBlock:
    def __init__(self, d_model, d_ff):
        self.d_model = d_model
        self.d_ff = d_ff

        self.ffn_w1 = np.random.rand(d_model, d_ff) * 0.1
        self.ffn_b1 = np.random.rand(1, d_ff) * 0.1

        self.ffn_w2 = np.random.rand(d_ff, d_model) * 0.1
        self.ffn_b2 = np.random.rand(1, d_model) * 0.1

    def feed_forward(self, x):

        hidden = np.maximum(0, np.matmul(x, self.ffn_w1) +
self.ffn_b1)
        output = np.matmul(hidden, self.ffn_w2) + self.ffn_b2
        return output

    def call(self, x):

        attention_output, _ = scaled_dot_product_attention
(x, x, x)

        # Residual connection
        attention_output = attention_output + x
```

```

# Feed-forward network
ffn_output = self.feed_forward(attention_output)
# Residual connection
ffn_output = ffn_output + attention_output

return ffn_output

```

Finally, we can test the encoder block using a random input tensor.

```

np.random.seed(42)

batch_size = 1
seq_len = 5

d_model = 64
d_ff = 128

x = np.random.rand(batch_size, seq_len, d_model)
transformer_block = SimpleTransformerBlock(d_model, d_ff)

output = transformer_block.call(x)

print("Input shape:", x.shape)
print("Output shape:", output.shape)

print(output[0, :2, :5])

```

Input shape: (1, 5, 64)

Output shape: (1, 5, 64)

[[20.27385289 20.49456842 21.77016518 20.93954099 22.34990271]

[19.80365965 20.27802678 21.76127071 20.23812619 21.24099152]]

The output has the same shape as the input because each token is transformed into a richer contextual representation rather than changing the sequence length.

Although this implementation is greatly simplified, it demonstrates the essential workflow of a Transformer encoder:

1. Each token attends to every other token using self-attention.
2. Residual connections preserve useful information.
3. A feed-forward network further refines each token representation.
4. Multiple encoder blocks can then be stacked together to produce increasingly powerful contextual embeddings.

In production Transformer models, these operations are repeated many times, allowing the network to capture increasingly complex semantic and syntactic relationships throughout the input sequence.

Summary

In this section, we introduced the **Transformer**, the architecture that revolutionized Natural Language Processing by replacing recurrent computation with **self-attention**. Unlike RNNs, Transformers process all tokens in parallel while effectively capturing long-range dependencies.

We explored the major components of a Transformer, including **tokenization**, **embeddings**, **self-attention**, **multi-head attention**, **residual connections**, **layer normalization**, and **positional embeddings**, all of which work together to produce rich contextual representations of text.

Finally, we implemented the core building blocks of a Transformer Encoder, including **scaled dot-product attention** and a simplified **encoder block**, laying the foundation for building modern NLP models such as sentiment classifiers, machine translators, and large language models.

References

<https://www.geeksforgeeks.org/machine-learning/getting-started-with-transformers/>

<https://machinelearningmastery.com/the-transformer-model/>

<https://www.ibm.com/think/topics/transformer-model>

Next Section:

 [17.2. The Sequence-to-Sequence Models](#)